

1. Given an integer array `nums` sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same. Return `k` after placing the final result in the first `k` slots of `nums`. Do not allocate extra space for another array; you must do this by modifying the input array in-place with $O(1)$ extra memory. No need to return the array.

Example 1

Input:

`nums = [1,1,2]`

Output:

2, `nums = [1,2, _]`

Example 2

Input:

`nums = [0,0,1,1,1,2,2,3,3,4]`

Output:

5, `nums = [0,1,2,3,4,_,_,_,_,_]`

2. Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

Example 1:

Input: `nums = [1,3,5,6]`, `target = 5`

Output: 2

Example 2:

Input: `nums = [1,3,5,6]`, `target = 2`

Output: 1

Example 3:

Input: `nums = [1,3,5,6]`, `target = 7`

Output: 4

3. Given a triangle array, return the minimum path sum from top to bottom. For each step, you may move to an adjacent number of the row below. More formally, if you are on index `i` on the current row, you may move to either index `i` or index `i + 1` on the next row.

Example 1:

Input: `triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]`

Output: 11

Explanation: The triangle looks like:

```
  2
 3 4
6 5 7
4 1 8 3
```

The minimum path sum from top to bottom is $2 + 3 + 5 + 1 = 11$.

Example 2:

Input: `triangle = [[-10]]`

Output: -10

4. Given a pattern and a string `s`, find if `s` follows the same pattern. Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in `s`. Specifically:

Each letter in pattern maps to exactly one unique word in s.
Each unique word in s maps to exactly one letter in pattern.
No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

'a' maps to "dog".

'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

5. Given a string s, return the number of segments in the string. A segment is defined to be a contiguous sequence of non-space characters.

Example 1:

Input: s = "Hello, my name is John"

Output: 5

Explanation: The five segments are ["Hello,", "my", "name", "is", "John"]

Example 2:

Input: s = "Hello"

Output: 1